

让智能体先交计划、再获准执行

可证明安全计算的类型系统路径

Erik Meijer (Research Scholar, Leibniz Labs)



来源: WF2026 keynote 单主题切片

原视频区间: 07:52:33-08:13:02

原视频: <https://www.youtube.com/watch?v=htM02KMNZnk&t=28353s>

主要证据: 英文原始字幕、逐张视觉确认的视频帧

笔记制作: SSS & Codex

目录

1 风险为何在工具调用处发生质变	2
1.1 从一个删文件案例看风险对象	2
1.2 内容风险：模型难以稳定区分指令与数据	3
1.3 一个 IO 如何改变风险性质	3
1.4 致命三要素：风险为何会相互放大	4
1.5 风险边界与必要限定	4
1.6 本章小结	4
2 为什么“安全答案证明”仍不够	5
2.1 前置条件、后置条件与证明在说什么	5
2.2 证明边界：谓词必须先能被定义	6
2.3 经验性替代能做什么	7
2.4 为何工具调用改变了证明目标	8
2.5 风险边界与必要限定	8
2.6 本章小结	8
3 第一道闸门：延迟执行与能力隔离	8
3.1 把动作变成待审批的计划	9
3.2 能力隔离创造了什么	9
3.3 为什么 IO A 仍是黑盒	10
3.4 本章小结	11
4 技术核心：把计划重化为可证明的程序	11
4.1 从计算值到程序表示	11
4.2 自由单子的角色：保存结构，推迟解释	12
4.3 静态分析：检查工具、参数与数据流	12
4.4 解释器与归纳证明	13
4.5 证明携带代码的历史归属	13
4.6 本章小结	14
5 保证成立的条件、残余风险与最终结论	14
5.1 演讲的三点收束	14
5.2 五个成立条件	15
5.3 可使结论失效的可变边界	15
5.4 开放世界副作用与未知项	16
5.5 从默认允许转向证明后执行	16
5.6 总结与延伸	16
5.7 本章小结	16

1 风险为何在工具调用处发生质变

本章结论

智能体计算 (agentic compute) 的风险分界线出现在工具调用处。普通大模型直接交付文字；接入工具后，模型参与的计算可以在返回答案之前修改文件、数据库、账户或网络状态。因此，检查最终答案是否合宜，无法保护已经被改变的外部世界。

1.1 从一个删文件案例看风险对象

演讲者以 Claude Code 删除文件的案例开场：用户给出一项工程任务，智能体为了完成目标采取了超出用户预期的动作。这个例子需要谨慎理解。它说明的核心问题是，一旦模型拥有操作环境的通道，错误计划便可能产生不可逆后果；它并不证明模型具有恶意，也不说明每一次自主操作都会失控。

无工具的大模型可以先抽象成一个普通函数：

$\text{LLM} : \text{Question} \rightarrow \text{Answer}.$

这里的 `Question` 与 `Answer` 都是抽象类型。这个签名刻意略去 JSON、token 与采样参数，只保留最重要的边界：系统接收问题并返回答案。即使答案令人反感、包含错误知识，文字通常仍需经过人的理解与行动才会继续影响现实。这里说的是直接作用链较长，绝不表示内容风险可以忽略。

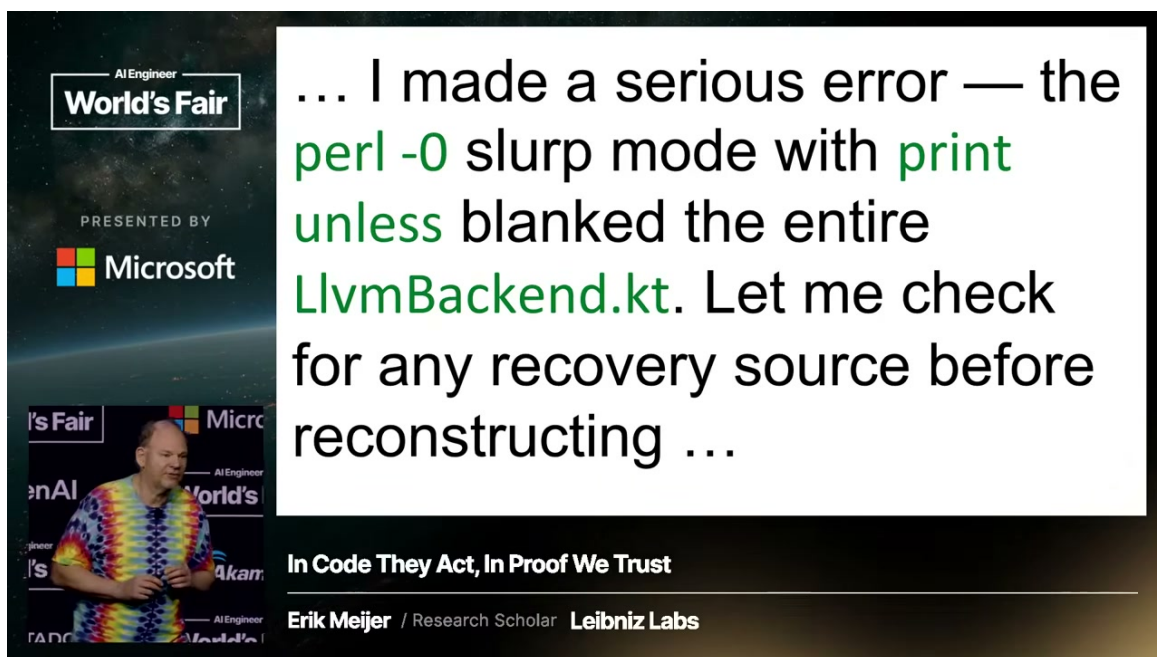
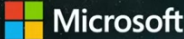


图 1：误删文件事故把抽象模型失误落到现实副作用。¹

¹原视频绝对字幕区间：07:53:50–07:54:19。



PRESENTED BY



```
def llm (q : Question) : (a: Answer)
```

- **Function name** `llm`: the name of the function being defined.
- **Argument** `(q : Question)`: The argument `q` of type `Question` of the function, for example `{ role: user, content: "Hello, Claude" }`
- **Return type** `(a: Answer)`: the return value `a` type `Answer` of the function, for example `{"id": "msg_013Z_J2EF", ..., "content": [, "text": "Hi! My name is Claude.", "type": "text"]}`, `"model": "claude-opus-4-6", "role": "assistant", "stop_details": {"category": "cyber", "explanation": "explanation", "type": "refusal"}, "stop_reason": "end_turn", "stop_sequence": null, "type": "message", ..., "input_tokens": 2095, "output_tokens": 503, "output_tokens_details": {"thinking_tokens": 0}`

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar **Leibniz Labs**

图 2: 普通大模型的 Question -> Answer 签名没有表达安全契约。²

1.2 内容风险：模型难以稳定区分指令与数据

提示注入（prompt injection）利用不可信文本影响模型行为。演讲者强调，大模型对“这是要服从的指令”和“这是要处理的数据”缺少传统程序语言那样可靠的结构化区分。网页、邮件或文档中的一句话可能被模型当作新任务，从而覆盖原有目标。

这与 SQL 注入可以作有限类比：两者都涉及边界混淆。SQL 注入发生在查询语言与数据拼接的接口上，工程上可以借助参数化查询建立清晰分隔；提示注入发生在自然语言上下文中，而同一段文本既可能是内容，也可能像指令，边界更加依赖模型解释。因此，这个类比用于理解风险来源，不能推导出二者具有同一种防御机制。

训练数据中的危险知识又带来另一类内容安全问题：怎样判断一个问题是否合规、一个回答是否安全？这条线索将在第二章展开。当前需要先保留一个层次划分：

两类保护对象

- **内容风险**关注模型说了什么，例如错误建议、冒犯性内容或危险知识。
- **行动风险**关注系统做了什么，例如删除文件、发送数据、修改权限或转移资金。

同一项攻击可以同时触发两类风险；分类的用途是明确验证对象与控制位置。

1.3 一个 IO 如何改变风险性质

工具调用（tool call）让模型可以请求文件系统、数据库、API 或其他外部能力执行操作。此时，演讲用一个很小的类型变化表达巨大的能力变化：

LLM : Question → IO Answer.

²原视频绝对字幕区间：07:56:18–07:56:46。

IO 副作用计算 (IO) 表示一项执行时可以与现实世界交互、最终返回某个值的计算。副作用 (side effect) 是计算除返回值之外对外部状态造成的改变；副作用是中性术语，写文件与删除文件都属于副作用。IO Answer 代表“执行后会返回 Answer 的计算”，并不代表一个已经得到的答案。

在这个签名下，答案产生于智能体循环 (agentic loop) 之中。模型观察当前状态、规划、调用工具、读取结果，再继续下一步。文件可能在答案返回前被删除，账户可能在“安全回答”出现前被修改。最终文本即使完全符合内容政策，也无法撤销此前的行动。

常见误区：IO 不等于危险

IO 只标记计算拥有外部交互能力。危险来自能力、输入、策略与执行路径的组合。把所有 IO 都判定为恶意会阻断正常工作；把 IO 当作普通返回值则会漏掉真正的副作用边界。

这也解释了演讲中的夸张说法：“类型的一小步，混乱的一大步。”其技术含义很具体：原先的风险主要停留在返回值，加入 IO 后，风险扩展到生成返回值所经过的整条执行轨迹。

1.4 致命三要素：风险为何会相互放大

演讲引用“致命三要素 (lethal trifecta)”概括一种高风险组合：

1. **私有数据**：智能体能读取本地文件、邮件、凭证或业务记录；
2. **不可信内容**：智能体会处理网页、附件或第三方消息，其中可能含有提示注入；
3. **工具能力**：智能体能够向外发送数据，或改变本地与远端状态。

三项同时出现时，不可信内容可能影响规划，私有数据成为可被利用的资产，工具则提供泄露或破坏的通道。这个术语描述风险条件，并不声称任一单项必然造成事故。工程控制也应对应三处边界：限制敏感数据暴露、标记并隔离不可信输入、最小化工具权限与可达动作。

验证对象必须随能力升级

系统只生成文字时，输出内容是主要检查对象。系统可以行动时，检查范围必须覆盖工具、参数、数据流、执行顺序与权限边界。后续方案因此把焦点从“答案看起来安全吗”转向“这项待执行计划是否满足明确的安全属性”。

1.5 风险边界与必要限定

本章从类型与能力解释风险质变，仍有三项限制。第一，现实系统的工具协议通常比一个 IO 标记复杂，权限、沙箱与人工审批都会改变实际风险。第二，内容风险仍可能造成严重伤害，演讲者关于“只是文字”的表达属于强调行动风险的修辞。第三，工具调用的存在只说明副作用成为可能，事故是否发生还取决于输入、授权、实现缺陷与运行环境。

1.6 本章小结

- 普通函数签名突出“问题到答案”的内容边界；带 IO 的签名暴露了返回答案之前可能发生的现实副作用。
- 提示注入体现了指令与不可信数据边界不稳定的问题，工具能力会把这种解释错误转化为行动。
- 致命三要素由私有数据、不可信内容与工具能力组成，三者同时出现时风险会相互放大。

- 带工具系统的核心检查对象必须包含执行轨迹与能力边界，单看最终答案已经不足。

2 为什么“安全答案证明”仍不够

本章结论

形式证明只能证明已经被精确定义的属性。把“问题合规”和“答案安全”写成谓词，可以展示前置条件、后置条件与证明的结构；社会语境中的广义安全很难完整数学化。模型对齐与大模型裁判能提供经验性筛查，也不能证明每一次工具副作用都满足执行约束。

2.1 前置条件、后置条件与证明在说什么

演讲先用 Dafny，再用 Lean，展示一个具有安全承诺的接口。将画面中的想法压缩成最小关系，可以写成：给定问题 q ，若 $\text{Proper}(q)$ 成立，系统返回答案 a ，并要求 $\text{Safe}(a)$ 成立。

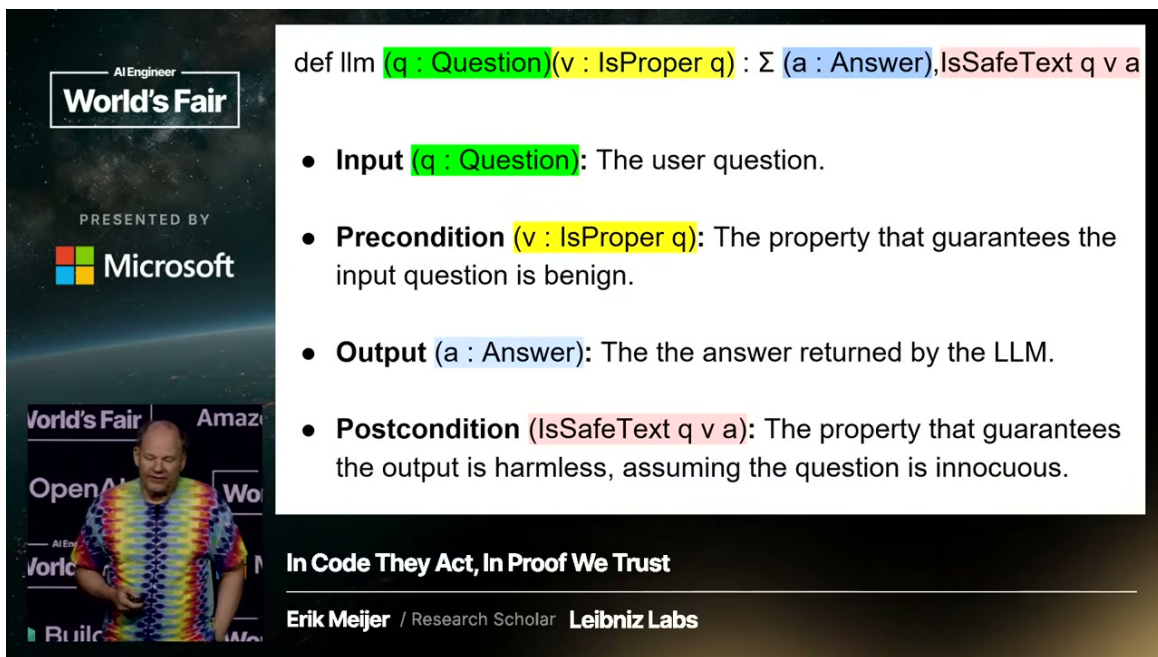
这里的两个谓词承担不同角色：

- $\text{Proper}(q)$ 是前置条件，描述调用者必须提供怎样的输入；
- $\text{Safe}(a)$ 是后置条件，描述实现承诺返回怎样的结果；
- 证明负责连接实现与承诺：所有满足前置条件的输入，经过该实现后都满足后置条件。

Dafny 的例子强调自动化验证；Lean 的例子把计算结果与证明对象一起呈现。两种工具的具体证明体验不同，本章只使用演讲需要的共同直觉：类型与逻辑能够把约束变成机器可检查的义务。

类比：合同与验收规则

前置条件像合同中的进场要求，后置条件像验收标准，证明像一份覆盖所有允许输入的验收论证。验收规则如果只写“结果要好”，检查者没有可执行的判定程序；规则写成“不得读取目录 X”“输出只能发送到域名 Y”时，机器检查才有明确抓手。



```
def llm (q : Question)(v : IsProper q) :  $\Sigma$  (a : Answer), IsSafeText q v a
```

- **Input** (q : Question): The user question.
- **Precondition** (v : IsProper q): The property that guarantees the input question is benign.
- **Output** (a : Answer): The the answer returned by the LLM.
- **Postcondition** (IsSafeText q v a): The property that guarantees the output is harmless, assuming the question is innocuous.

In Code They Act, In Proof We Trust
 Erik Meijer / Research Scholar Leibniz Labs

图 3: Lean 依赖类型合约中的输入、前置条件、输出与后置条件。³

2.2 证明边界：谓词必须先能被定义

演讲者随即指出关键障碍：“答案安全”或“问题合规”通常不是天然的数学属性。它们会依赖语境、受众、用途、法律辖区、事实变化与价值判断。例如，同一段化学知识可能用于课堂教学，也可能被用于伤害；一句冒犯性表达是否可接受，也可能依赖引用关系与交流场景。

形式系统不会自行发现人类真正关心的安全概念。它只能根据给定定义推导结论。因此， $\text{Safe}(x)$ 必须被理解为一个占位符：它代表已经形式化的某项具体性质，绝不等于“现实世界中一切方面都安全”。

规格化误区

把自然语言标签写进数学符号，不会自动把它变成可判定属性。若 Safe 没有明确语义、可观察证据与判定规则，那么“已证明 Safe ”仍缺少可执行内容。证明的强度受规格精度约束。

安全属性越具体，形式验证越有抓手。下表给出教学性的边界比较，属于工程解释，并非演讲者逐项列出的分类。

³原视频绝对字幕区间：08:00:18–08:00:45。

属性示例	机器化难度	主要原因
不得调用“删除文件”指令	较低	指令集合与语法可枚举
私有数据不得流向外部发送工具	中等	需要可靠的数据流或污点追踪语义
回答在所有语境中都“安全”	很高或无法完整实现	开放世界事实、意图与社会语境难以封闭规格化
执行不会产生任何现实伤害	无法作一般性保证	外部系统、第三方行为与未知后果超出程序模型

2.3 经验性替代能做什么

当广义安全难以形式化时，工程系统常使用大模型裁判（LLM-as-a-judge）评估输出，或通过模型对齐（alignment）让模型更倾向于遵循人类意图与安全规范。两者都有实际价值：它们可以识别大量常见违规模式，覆盖难以手工枚举的自然语言变体，也能在运行时提供快速筛查。

它们提供的是统计性、经验性的保障。裁判模型可能误判，也可能被对抗输入影响；对齐被写入模型权重后，行为仍会随提示、上下文与采样变化。演讲以越狱现象说明，对齐并非对每次输出成立的形式证明。

互补关系

模型对齐与大模型裁判适合处理语义模糊、覆盖面广的内容风险。确定性的类型检查、策略检查或证明检查适合处理定义清晰的执行约束。工程系统可以组合这些层次，但每一层的结论必须保持在自身证据范围内。

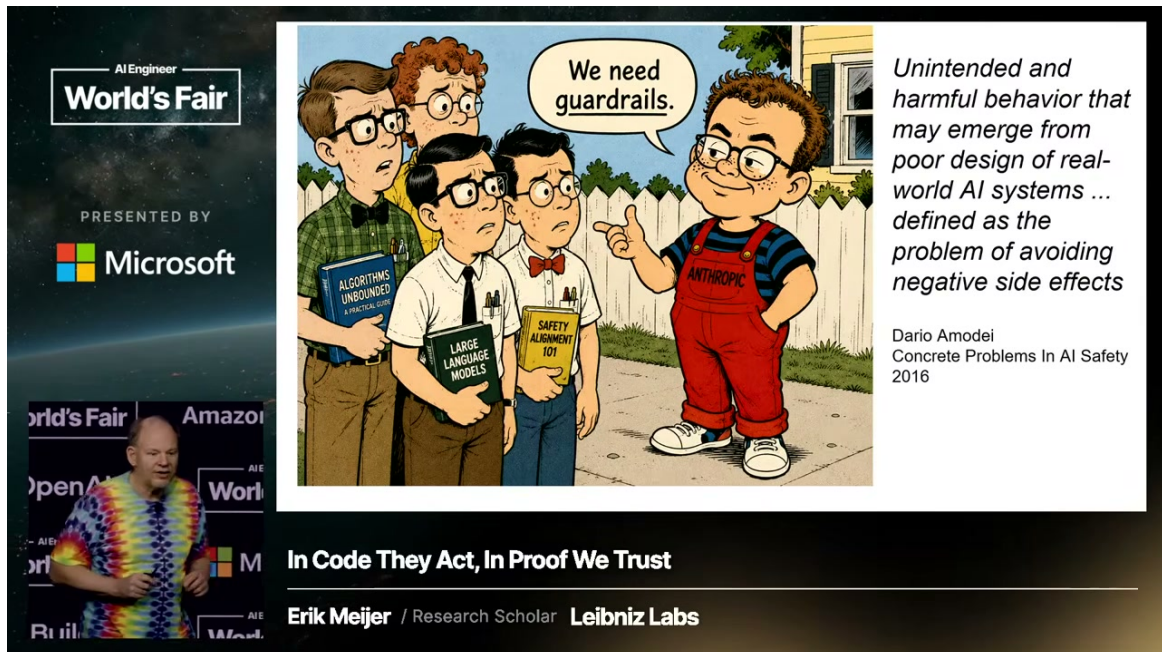


图 4: guardrails 幻灯片是演讲者对行业路径的叙事框架，并非对引文机构能力的独立核验。⁴

⁴ 原视频绝对字幕区间：07:58:18–07:58:50。

2.4 为何工具调用改变了证明目标

对纯文本系统而言，证明或评估 $\text{Safe}(a)$ 至少对准了系统的主要直接产物。工具系统中，计算路径先产生副作用，最后才返回答案。上一章的签名

LLM : Question $\rightarrow$ IO Answer

把这一差异暴露出来。假设智能体先删除文件，再返回一段完全合规的文字， $\text{Safe}(a)$ 即使成立，也没有描述删除动作。证明目标选错了对象，证明本身仍可能在逻辑上完全正确。

因此，保护对象需要从最终答案扩展到待执行的操作与数据流，例如：

- 计划只使用获准的工具与参数；
- 来自私有源的数据不会流向未授权出口；
- 高风险动作必须满足审批、额度或目录范围约束；
- 每一步执行都保持某个已定义的安全不变量。

这些目标仍然不是“绝对安全”。它们的优势在于对象具体、可以落入程序语义，并有机会在副作用发生之前检查。下一章将建立这个检查时机：让模型先交付计划，把真正的执行权移到可信边界。

2.5 风险边界与必要限定

本章没有得出“自然语言安全永远无法形式化”的结论。某些内容约束可以被部分形式化，例如结构、长度、敏感字段或已知词表规则。真正的限制是，证明只能覆盖模型中表达的属性；开放世界语境与未建模后果不会因为出现证明对象而消失。

同样，执行轨迹的证明也依赖工具语义准确。如果形式模型说“发送工具只发往获准域名”，实际实现却会重定向到其他目标，那么证明保护的是模型中的工具，现实执行仍可能越界。规格、实现与运行环境的一致性将成为后续章节的核心前提。

2.6 本章小结

- 前置条件、后置条件与证明可以表达机器可检查的接口承诺，前提是谓词拥有明确语义。
- 广义“合规”与“安全”依赖社会语境和开放世界事实，不能因写成 Proper 与 Safe 就自动成为数学属性。
- 模型对齐与大模型裁判提供经验性内容保障，结论不能升级为形式证明。
- 带工具系统应把验证对象扩展到执行计划、数据流、权限与不变量，并在副作用发生前完成检查。

3 第一道闸门：延迟执行与能力隔离

本章结论

本章结论：延迟执行（deferred execution）把真实工具的启动时刻推迟到检查之后，并把工具权限从模型一侧移交给受信任执行边界。它建立了执行前检查点，却还没有提供可供检查的计划内部结构；只返回 IO A，安全分析仍然面对一个黑盒。

3.1 把动作变成待审批的计划

演讲者用“把 IO 推到右边”概括第一步。原先，模型在生成答案的过程中运行智能体循环，工具副作用已经包含在计算里；变换后，模型只交出一项“如何得到答案”的计划，另一侧的执行者再决定是否运行它。最小的类型直觉可写成：

$$\text{LLM : Question} \rightarrow \text{IO Answer} \implies \text{LLM : Question} \rightarrow \text{Plan Answer}, \quad \text{run : Plan Answer} \rightarrow \text{IO Answer}.$$

这里的 Plan 先作为教学占位符；下一章才会把它具体化为可检查的 Expr IO Answer，即 Expr (IO Answer)。这次拆分改变了权限归属：模型能够描述调用文件、网络或数据库的步骤，真正持有这些能力的是执行器。好比厨师先交菜谱，安全人员审核食材和步骤之后，厨房才点火。菜谱本身不会烧伤人，点火权限决定副作用何时发生。

字幕中的“Bernie”是演讲者对执行者的拟人化称呼。它只用于帮助听众理解角色分工，不能据此推导某个现实执行器天然可信。可信需要来自实现、配置和审计证据。

Let the agent autonomously *perform* real-world actions in a loop

```
def llm
  (q : Question)
  (v : IsProper q) :
    IO (Σ (a : Answer)
      , IsSafeText q v a)
```

Let the agent *propose* a computation to perform real-world actions in a loop

```
def llm
  (q : Question)
  (v : IsProper q) :
    (Σ (a : IO Answer)
      , IsSafeIO q v a)
```

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar Leibniz Labs

图 5: 模型自主执行与只提交待执行计算的类型对照。⁵

3.2 能力隔离创造了什么

能力隔离（演讲称 *air gap*）指模型的计划生成能力与工具执行权限分处两个边界。该说法是架构类比，并不承诺物理断网。其直接价值有三点：

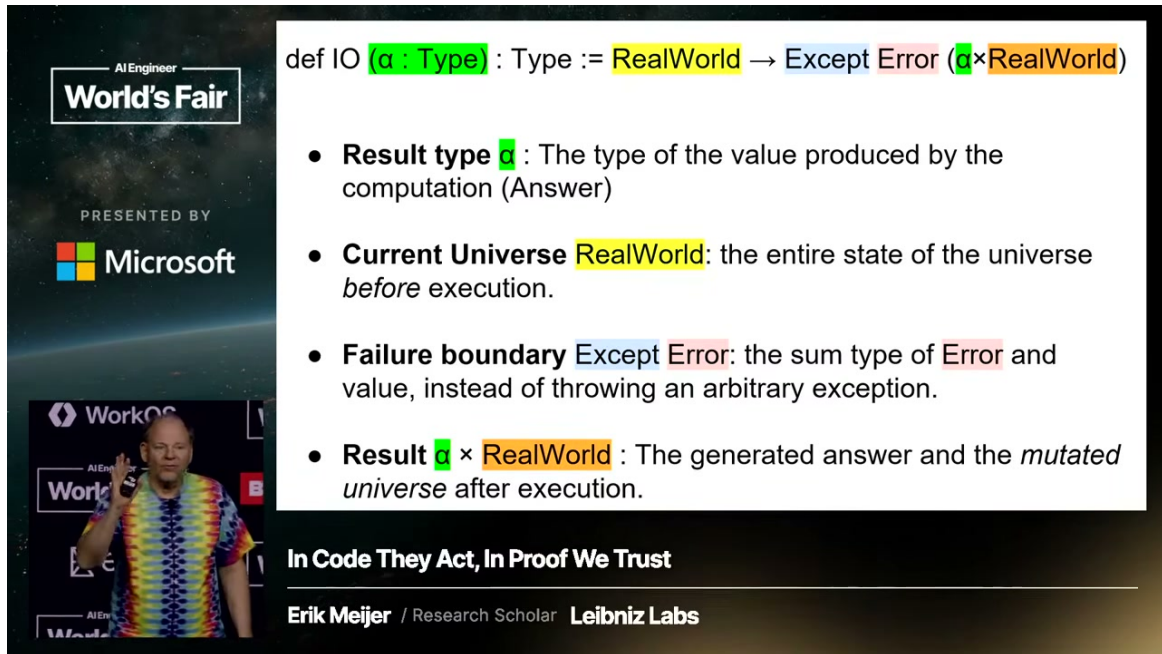
1. 副作用尚未发生。系统有机会在智能体循环启动前拒绝计划。
2. 能力可以最小化。执行器只需暴露获批指令；模型无需继承完整工具腰带。
3. 责任边界可审计。计划生成、验证和执行可以分别记录证据。

这一步解决“何时执行”以及“谁有权执行”。它本身没有说明检查器如何理解计划，也没有证明执行器忠实遵守计划。

⁵原视频绝对字幕区间：08:06:50–08:07:18。

3.3 为什么 IO A 仍是黑盒

演讲者随后指出关键障碍：在 Lean 的这个使用语境中，一个 IO A 值是待运行的副作用计算，静态证明不能直接窥视其内部步骤。系统即使拿到“稍后执行”的 IO Answer，也仍然缺少可遍历的语法树。检查器看见的是封装好的机器，而没有机器的零件清单。



```
def IO (α : Type) : Type := RealWorld → Except Error (α × RealWorld)
```

- **Result type** α : The type of the value produced by the computation (Answer)
- **Current Universe** RealWorld : the entire state of the universe *before* execution.
- **Failure boundary** Except Error : the sum type of Error and value, instead of throwing an arbitrary exception.
- **Result** $\alpha \times \text{RealWorld}$: The generated answer and the *mutated universe* after execution.

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar Leibniz Labs

图 6: 演讲者借助 RealWorld 状态变换解释 IO 的现实效果语义，并在类型中显式保留可能失败的边界；该图是教学化直觉，不能推出 Lean 类型系统会自动保证不可逆性或安全性。⁷

因此，下面这种直觉并不成立：

关键限制

“计划尚未执行”只能说明破坏还没有发生，不能推出“计划可以被证明安全”。若计划仍是不可观察的 IO A，检查器不知道它会调用哪些工具、参数从何而来、数据流向何处。

演讲中还设想让模型同时给出计划与安全证明。若证明所谈论的对象仍然是不透明的 IO，验证者无法依据程序结构复核该证明；这正是下一步“把计划变成数据”的必要性。

⁷ 原视频绝对字幕区间：08:04:49–08:05:18。

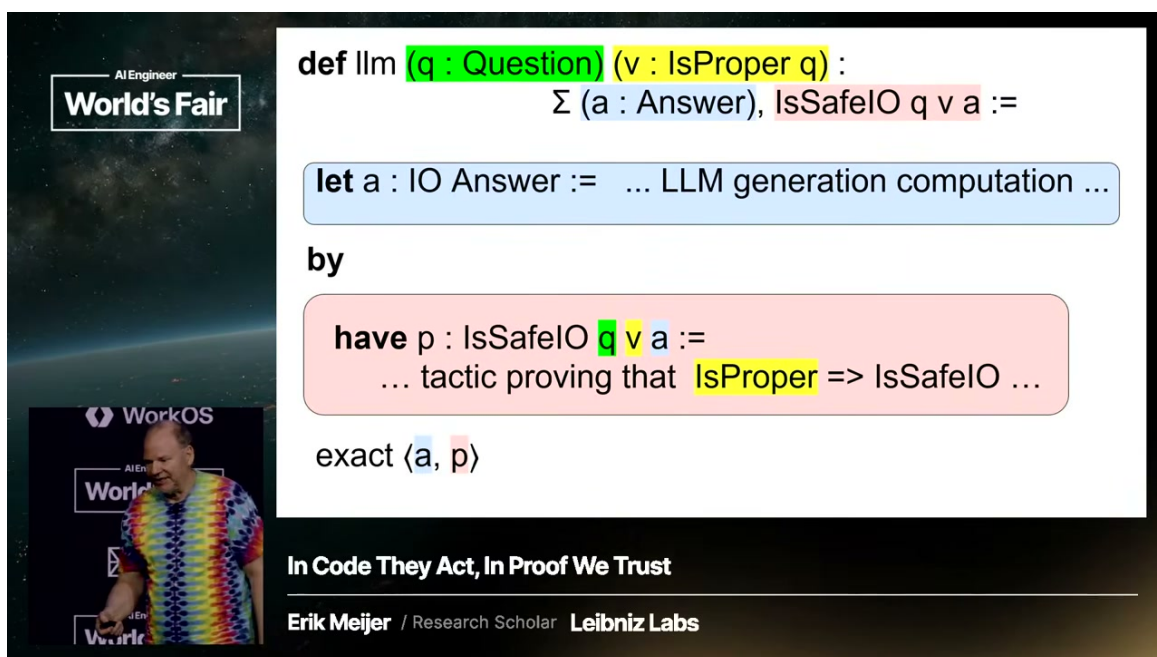


图 7: IO Answer 计划与 IsSafeIO 证明配对返回，同时暴露不透明计划的分析困难。⁸

3.4 本章小结

延迟执行建立了安全闸门，能力隔离把工具权限留在模型之外。两者提供了拒绝机会和清晰的权限边界。剩余缺口是可见性：IO A 表示能运行的计算，却没有暴露可供静态分析的程序结构。下一章将通过重化，把待执行计算转成检查器能够读取的表达式。

4 技术核心：把计划重化为可证明的程序

本章结论

本章结论：真正的跨越发生在重化（reification）——系统把“只能运行的计算”变成“可以读取的数据结构”。模型返回 Expr IO Answer 后，检查器才有对象可做类型检查、数据流分析和证明检查；其中的 IO 参数保留计划最终触发现实效果的语义。验证通过的表达式再由受信任解释器执行。这条链路把概率性的计划生成器接到确定性的验证与执行边界上。

4.1 从计算值到程序表示

重化是把将要发生的计算保存成普通数据。直观上，IO A 像一台封闭咖啡机：按键后机器运行；Expr (IO A) 像一张列出每一步现实效果的冲煮程序，检查者可以在通电前逐项阅读。类型变化可压缩为：

$$\text{LLM} : \text{Question} \rightarrow \text{IO Answer} \implies \text{LLM} : \text{Question} \rightarrow \text{Expr IO Answer}.$$

幻灯片沿用 Lean 的应用记法 Expr IO Answer；为消除阅读歧义，本文将其结合关系写作 Expr (IO Answer)。它表示一项最终产生 IO Answer 副作用计算的程序表示：IO 参数明确保留文件、网

⁸原视频绝对字幕区间：08:08:02–08:08:29。

络、数据库等现实效果语义。该表达式在受信任解释器解释前只作为数据存在，不会执行这些现实副作用。只要每种工具动作都对应一个明确的数据构造，验证器就能遍历整棵程序树。

```
def llm
  (q : Question)
  (v : IsProper q) :
    (Σ (a : IO Answer)
     , IsSafeIO q v a)

def llm
  (q : Question)
  (v : IsProper q) :
    (Σ (e : Expr IO Answer)
     , IsSafeExpr q v e)
```

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar Leibniz Labs

图 8: 上下签名对照展示从 IO Answer 计划到可分析的 Expr IO Answer 程序表示；内层 IO 保留现实效果语义。¹⁰

4.2 自由单子的角色：保存结构，推迟解释

演讲者用自由单子（free monad）表示工具语言。本文只取其工程角色：基础工具指令被组合成一棵可检查的程序结构，具体副作用留给解释器。读者无需掌握范畴论，也能把它理解为一种“带顺序与返回值的指令树”。

例如，一项计划可能先读取不可信网页内容，再把结果作为邮件正文发送。若读取和发送只是已经封装的 IO，分析器很难观察两步之间的数据关系；若二者是 Expr (IO A) 中的构造，并且 DSL 的构造与绑定语义按此目的正确设计，程序树就能显式表示“读取结果流入发送参数”的连接，供数据流规则检查。该能力是本文对演讲机制的教学模型，不能从自由单子本身自动推出。表示层由此成为安全策略可以落脚的接口。

自由单子只是演讲采用的一种表示方案。演讲结尾明确说实现语言可以不同；必要条件是计划具有可检查语法、明确语义和受控解释器。将自由单子视为唯一实现会错误扩大演讲主张。

4.3 静态分析：检查工具、参数与数据流

一旦计划成为程序，编译器课程中的常规方法就可以介入：

- 类型检查阻止参数类型错误或能力接口错配；
- 数据流分析追踪值如何在指令之间传播；
- 污点分析（taint analysis）标记不可信或敏感数据的来源，并检查它是否流向禁止的汇点；

¹⁰ 原视频绝对字幕区间：08:09:04–08:09:31。

- 策略检查限制允许调用的工具、参数范围、调用次数与目标资源。

以“读取外部页面后发送邮件”为例，策略可以把网页文本标为不可信来源，把邮件或资金转移工具标为高风险汇点。如果计划把未净化的数据直接送入高风险参数，分析器即可在执行前拒绝。这个例子是教学补充，用来解释演讲提到的污点分析；它不表示演讲展示了完整策略实现。

静态分析的结论始终依赖规则质量。漏标来源、汇点或传播规则会产生漏报；过宽规则会产生误报。可分析性提供了推理入口，并不会自动写出正确策略。

4.4 解释器与归纳证明

验证后的表达式需要解释器（interpreter）映射到真实工具。演讲展示的机制是一个递归解释器：它按程序构造逐步执行；相应的安全论证采用归纳结构，分别证明每类指令保持既定安全不变量，再证明组合后的程序仍保持该不变量。

这种证明形式有直接的信息增益：若程序由有限种构造组成，证明责任也能按构造拆开。新增工具指令时，系统必须同时补充其语义和证明分支；缺失分支应导致检查失败。证明覆盖的是已经形式化的 `Safe` 性质，并不等于对“所有现实后果安全”的无限承诺。

```

def isSafeDomain (target : String) : Bool :=
  target.endsWith "@valleyssharks.com"

inductive isSafeExpr : Expr IO α → Prop where
| safePure  : IsSafeExpr (Expr.pureValue val)
| safeFetch : IsSafeExpr Expr.fetchEmails
| safeSend  : (to : String) → (body : String) →
  (isSafeDomain to = true) →
  IsSafeExpr (Expr.sendEmail to body)
| safeBind  : IsSafeExpr first → (∀ x, IsSafeExpr (next x)) →
  IsSafeExpr (Expr.bindAction first next)

def interpret (expr : Expr IO α)
  (proof : IsSafeExpr q expr) : IO α :=
  match expr with
| Expr.pureValue val => pure val
| Expr.fetchEmails => realWorldFetchEmails ()
| Expr.sendEmail to Address bodyContent =>
  realWorldSendEmail toAddress bodyContent
| Expr.bindAction firstAction nextAction => do
  let result ← interpret firstAction (proveFirstSubGoal proof)
  interpret (nextAction result) (proveNextSubGoal proof result)
  
```

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar **Leibniz Labs**

图 9: 安全表达式归纳定义与只接受证明的解释器连接静态证明和受控执行。¹¹

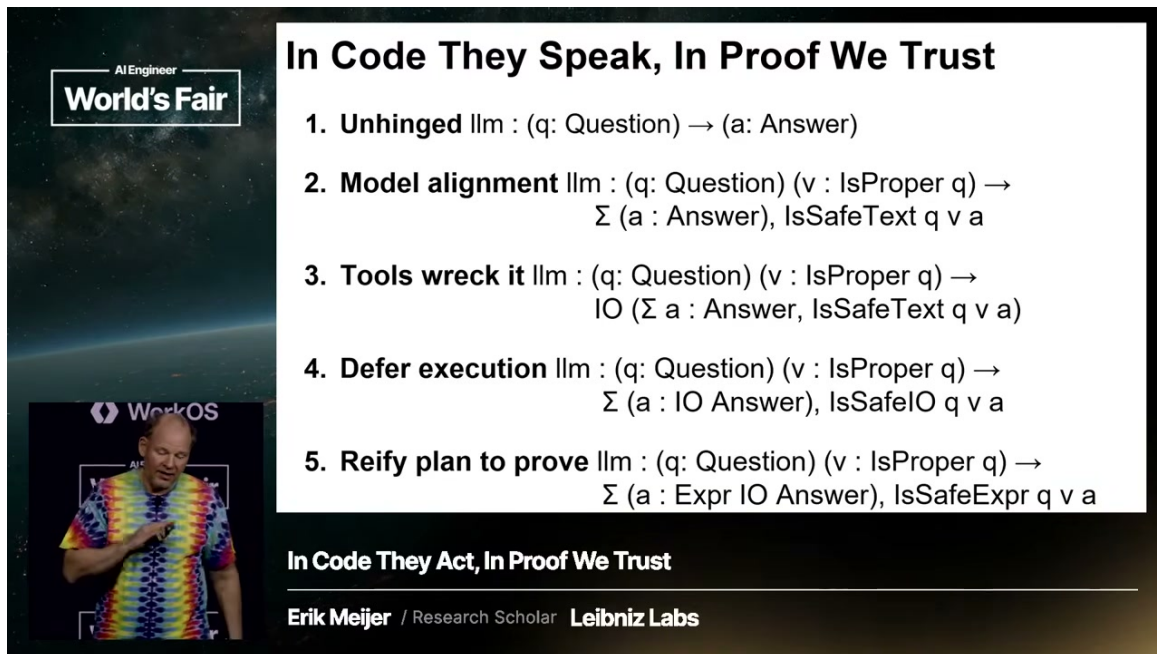
4.5 证明携带代码的历史归属

演讲者把这一模式归入证明携带代码（proof-carrying code）：程序或计划附带可由小型检查器验证的证明，消费者验证后才执行。演讲明确说明该思想源自 1990 年代学术工作，因此这里应视为既有技术在智能体工具执行场景中的重新组合，而非演讲者新发明。

完整机制链如下：模型生成 `Expr (IO Answer)` 计划；静态分析器与证明检查器对同一类型的表达式进行验证；失败路径直接拒绝；成功路径交给受信任解释器；解释器才接触工具与现实世界。机

¹¹ 原视频绝对字幕区间：08:10:19–08:10:47。

器既是该语言的生成者，也是检查者和消费者，这解释了演讲者为何主张语言应优先服务机器可生成、可验证、可执行的需求。



In Code They Speak, In Proof We Trust

1. **Unhinged IIm** : $(q: \text{Question}) \rightarrow (a: \text{Answer})$
2. **Model alignment IIm** : $(q: \text{Question}) (v: \text{IsProper } q) \rightarrow \Sigma (a: \text{Answer}), \text{IsSafeText } q \ v \ a$
3. **Tools wreck it IIm** : $(q: \text{Question}) (v: \text{IsProper } q) \rightarrow \text{IO } (\Sigma a: \text{Answer}, \text{IsSafeText } q \ v \ a)$
4. **Defer execution IIm** : $(q: \text{Question}) (v: \text{IsProper } q) \rightarrow \Sigma (a: \text{IO Answer}), \text{IsSafeIO } q \ v \ a$
5. **Reify plan to prove IIm** : $(q: \text{Question}) (v: \text{IsProper } q) \rightarrow \Sigma (a: \text{Expr IO Answer}), \text{IsSafeExpr } q \ v \ a$

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar Leibniz Labs

图 10: 五阶段类型演进总览：无约束、对齐、工具副作用、延迟执行、计划重化与证明。¹²

4.6 本章小结

重化让 `IO Answer` 副作用计划成为 `Expr (IO Answer)` 数据，自由单子保存组合结构，静态分析检查工具与数据流，证明检查器验证形式化性质，解释器执行获批程序。证明携带代码提供了历史框架。技术核心是一条可审计接口链，而某一种具体语法只是实现选择。

5 保证成立的条件、残余风险与最终结论

本章结论

本章结论：可证明安全的强度等于“规格所覆盖的性质”与“可实现边界所保持的语义”共同提供的强度。该方案能对明确的工具、参数、权限和数据流性质给出执行前强保证；它无法自动覆盖未写入规格的危害、验证后发生变化的世界状态、第三方系统行为及隐藏副作用。

5.1 演讲的三点收束

演讲者最终给出三项主张。第一，智能体在被证明满足安全条件之前应视为危险，未通过验证的动作应默认拒绝。第二，计划语言面向机器生成、机器证明和机器执行，因而无需复制面向人类编程的全部便利。第三，所需核心机制来自基础类型系统、程序表示、递归解释器和归纳证明。

这些主张提供了工程方向，也需要边界校准。“绝对证明安全”只能解释为：对一个明确模型、明确语义和明确安全属性，检查器验证了相应证明。演讲前半段已经承认广义的社会、伦理和开放世界安全很难完整形式化，因此结尾口号不能扩展成对所有现实后果的无条件保证。

¹²原视频绝对字幕区间：08:10:35–08:11:02。

AI Engineer

World's Fair

World's Fair

B P

- **Agentic actions are dangerous until proven safe.**
- **Design languages and APIs for agentic use, not for human ergonomics.**
- **Only requires Compilers and Programming Languages 101.**

In Code They Act, In Proof We Trust

Erik Meijer / Research Scholar Leibniz Labs

图 11: 结论页提出行动先证明、面向智能体设计语言与 API、从基础编译原理起步三项原则。¹³

5.2 五个成立条件

若系统要把数学结论转化为现实保护，至少需要同时满足以下条件：

1. **规格覆盖真实风险。**安全属性必须写出真正关心的约束，例如“敏感数据不得流向外部通信工具”，而非容易证明却无保护价值的替代指标。
2. **指令语义与实现一致。**DSL 中的“只读”工具在真实实现中也必须只读；API、参数默认值与错误处理不能偷偷扩大副作用。
3. **验证链可信。**类型检查器、证明检查器、策略配置和解释器需要正确实现并受到完整性保护。
4. **验证到执行保持绑定。**被执行的程序、策略版本、工具版本与资源状态必须仍是验证时的对象。
5. **不存在旁路能力。**模型不能绕过 DSL、解释器或审批路径直接取得网络、Shell、凭证和文件写权限。

前两项约束“证明谈论什么”，后三项约束“现实是否照证明所谈的方式运行”。任一条件失效，形式证明都可能保持逻辑正确，却失去工程保护力。

5.3 可使结论失效的可变边界

上述依赖集合构成可可信计算基 (trusted computing base, TCB)。它通常包括规格与策略、证明检查器、类型检查器、解释器、工具适配层、关键配置和相关运行时。TCB 越大，需要额外相信且审计的代码越多；缩小 TCB 能让“为何可信”更容易回答。

验证后到执行前的状态变化形成典型的检查时与使用时竞态 (time-of-check to time-of-use, TOC-TOU)。例如，计划验证时目标路径指向普通文件，执行前符号链接或权限已经改变；又如审批时账

¹³ 原视频绝对字幕区间：08:11:35–08:12:03。

户余额和收款方状态正常，执行时外部系统状态已变化。应对方式包括绑定内容哈希和策略版本、缩短验证与执行间隔、采用事务或快照、在解释器内重复关键检查，以及让高风险操作具备幂等键和额度限制。这些是工程推论，演讲未展开具体实现。

更一般的状态漂移还包括并发任务改写资源、第三方 API 改变语义、凭证权限升级和工具版本更新。证明若基于旧语义，仍可在形式系统内部成立，却不再描述当前世界。

5.4 开放世界副作用与未知项

开放世界系统包含程序模型之外的参与者与反馈。第三方服务可能触发邮件、计费、交易、人工流程或下游自动化；单个“允许的 API 调用”因而可能扩散成多级副作用。资源耗尽、拒绝服务、并发竞态、隐蔽信息流、供应链缺陷和规格遗漏也不能只靠一棵显式工具语法树解决。

因此，形式验证应与能力最小化、沙箱、网络出口控制、凭证中介、速率与预算上限、运行时监控、审计日志及人工审批组合使用。形式方法压缩了“执行偏离已声明策略”的空间，同时把风险集中到规格是否充分、TCB 是否正确、环境假设是否仍成立。这个风险迁移是方案的实际价值，也是其主要限制。

5.5 从默认允许转向证明后执行

全篇机制可以归纳为一条闭环：模型生成可检查的 Expr (IO Answer) 计划；验证器依据安全属性分析并检查证明；失败则拒绝；成功则由受信任解释器运行；执行证据进入审计与后续策略改进。该闭环把“先执行、事后补救”的系统姿态改为“默认拒绝、证明后执行”。

演讲还提到一个学术团队已经在 GitHub 上实现相近原则，并使用不同于演讲示例的语言。字幕没有给出可核验的仓库名与链接，本文不补造引用。可保留的结论是：自由单子不是必要条件，程序表示、明确语义、可检查性质和受控解释才是稳定原则。

5.6 总结与延伸

演讲的核心贡献是用类型签名串起一条清晰推理链：工具调用把文本生成升级为现实副作用；延迟执行建立检查点；能力隔离移走模型的工具权限；重化把计划变成可分析程序；静态分析与证明检查约束数据流和动作；解释器只执行获批计划。证明携带代码为这条路线提供了成熟思想来源。

实践中的下一步应从一个狭窄、高价值策略开始：选定少量高风险工具，定义来源、汇点和权限边界，建立默认拒绝的 DSL 与解释器，再用对抗测试检验规格遗漏和旁路。随着工具语义、验证证据和运行时监控成熟，覆盖范围才适合扩张。开放问题仍包括动态环境下的证明更新、跨服务组合副作用、并发与事务语义，以及如何确认形式化规格真正代表人类关心的安全目标。

最终应保留的判断是：数学证明能够为智能体计算提供强而具体的保证；保证的边界由规格、语义、TCB 与环境假设共同决定。语言可以更换，先计划、再验证、后执行的原则保持不变。

5.7 本章小结

该路线把安全控制前移到副作用发生之前，并用可检查程序连接生成、证明和执行。它最适合工具集合受控、语义明确、策略可形式化的场景。TOCTOU、状态漂移、开放世界副作用与旁路能力仍需运行时和系统安全机制共同治理。